Old Bailey Decisions: Final Report

Ritaja Das (u1373636)

I. Introduction

1. Project background:

The Old Bailey is the colloquial name for the Central Criminal Court of England and Wales, which deals with major criminal cases in Greater London and also sometimes from the rest of England and Wales. This court has existed in some form or another since the 16th century. The Proceedings of the Old Bailey are made up of 120 million words, recording 197,000 trials held at the Old Bailey, or Central Criminal Court in London, between 1674 and 1913. The proceedings of this court have been digitized and are available online in [1].
Since all the digitalized text of the trials from 1674 to 1913 are available, we can ask the following text classification questions:

- Is it possible to predict the decision of the court using the transcribed dialogue during a trial?
- If yes, how likely the prediction is correct?
- which classification algorithm performs the others?

The goal of this project is to answer these questions.

2. Dataset:

Each example for classification is a single trial. In a single trial, there may be more than one defendant, and more than one charge. To simplify things, this project has restricted the dataset to trials where there is exactly one defendant and one charge. As a result, each trial has exactly one outcome: either guilty (label 0) or not guilty (label 1). In this project in total 25,000 examples are randomly selected as a subset of the entire data, which originally consists of 197,000 examples. These examples have been split into three subsets as below:

- Train: This is the training split of the data, on which the models are trained after hyper-parameter selection. The training split consists of 17,500 examples.
- Test: This is the test set that are used to evaluate the models locally. The test set consists of 2,250 examples.

• Eval: This is the evaluation set with 5,250 examples. The labels for these examples are hidden. The trained classifiers should predict labels for this dataset and then the labels should be uploaded to Kaggle.

3. Feature representation of the Trials

The data has been pre-processed and four different feature sets are extracted as follows:

1) bag-of-words: The bag of words representation represents the text of the trial as a set of its words, with their counts. That is, each dimension (i.e feature) corresponds to a word, and the value of the feature is the number of times the word occurs in the trial. To avoid the dimensionality from becoming too large, we have restricted the features to use the 10,000 most frequent words in the data.

2) Term frequency–inverse document frequency: The term frequency–inverse document frequency (tfidf) is a popular document representation that seeks to improve on the bag of words representation by weighting each feature in the bag-of- words representation such that frequent words are weighted lower. As with the bag-of-words, we use the 10,000 most frequent words in the data.

3) Glove: The previous two feature representations are extremely sparse vectors, with only a small fraction of the 10,000 features being non-zero for any vector. The glove representation represents a document by the average of its "word embeddings", which are vectors that are trained to capture a word's meaning. The word embeddings are dense, 300 dimensional vectors, and for the purpose of this project, each word is weighted by its tfidf score.

4) Miscellaneous: In addition to the above-mentioned features, also miscellaneous categorical attributes are extracted from the trial data. This consists of the following features for each trial:

1) defendant age
2) defendant gender
3) number of victims
4) genders of the victims
5) offence category
6) offense subcategory

In order to overcome the project milestones I used Id3, Perceptron, Super vector machine (SVM), SVM over trees , Random Forest and neural network (NN).

II. Pre-processing of Data

Among the four feature representations of the dataset, three of them (bow, tfidf, glove) are already pre-processed and do not need to be processed further. However, the miscellaneous features are required to be pre-processed before being used for classification task. The miscellaneous dataset contains categorical variables. These variables are mainly text values which represent various traits. In order to be able to use them in classification they need to be converted to numerical dataset.

defendant_age	defendant_gender	num_victims	victim_genders	offence_category	offence_subcategory
Nineteen	male	1	male	theft	theftFromPlace
seventeen	male	1	male	theft	grandLarceny
sixteen	female	0	nan	kill	infanticide
eighteen	male	1	female	theft	grandLarceny
-46	male	0	nan	royalOffences	coiningOffences
nineteen	male	1	male	theft	animalTheft
seven	female	1	male	theft	grandLarceny

Table 1: misc data before pre-processing

III. Results and observations:

In this project, all of the classifiers and the pre-process are implemented in Python. I had to do pre-processing on the “libsvm” format and changed the labels to -1 and 1 so that the weights get updates correctly.

Purpose	File name
pre processing of misc data	pre_processing_misc.py

Table 2

Converts .csv file to .libsvm format.	csv2libsvm.py

Notations:
Kaggle: not included in final 6 submissions.
Kaggle: included in final submissions.

A. ID3:

I considered age as multiclass feature and applied ID3 on processed misc data set. No pre processing was needed for tfidf data set.

Algorithm	File Name	Accuracy on data set Misc	Accuracy on data set tfidf
ID3	Id3_misc.py	Accuracy on Train: 82 % Accuracy on Test: 77 % Kaggle: ID3_submission2.csv Accuracy on Eval: 76%	Accuracy on Train: 50 % Accuracy on Test: 48 % Kaggle: ID3_submission_tfidf.csv Accuracy on Eval: 50%

Table 3: ID3

```
training accuracy 0.8201142857142857
testing accuracy 0.7711111111111111
eval accuracy 0.8201142857142857
```

Figure 1: Id3 on misc

```
training accuracy 0.5022285714285715
testing accuracy 0.484
eval accuracy 0.5022285714285715
```

Figure 2: Id3 on tfidf

B. Average Perceptron:

Algorithm	File Name	Accuracy on data set tfidf
Average Perceptron	avg_perceptron_tfidf_submission1.py	Accuracy on Train: 79.94 % Accuracy on Test: 70.97% Kaggle: avg_perceptron_tfidf.csv Eval: 70%

Table 4: Avg perceptron

```
accuracy on training 0.7994857142857142
accuracy on testing  70.97777777777777
```

Figure 3: Avg perceptron on tfidf

C. Margin Perceptron:

I wanted to explore margin hyper-parameters along with the learning rate. I used BOW dataset. I used the max value of each column to scale the features.

Algorithm	File Name	Accuracy on data set bow

Margin Perceptron	margin_perceptron_bow.py	Accuracy on Train: 71.54 % Accuracy on Test: 69.51% Kaggle : margin_perceptron_bow.csv Eval: 63%

Table 5:Margin perceptron

```
Starting reading files for margin using bow
training accuracy 71.54285714285714
eval accuracy 41.523809523809526
testing accuracy 69.5111111111111
```

Figure 4: Margin perceptron

D. SVM

Algorithm	File Name & Kaggle submission	Accuracy on data set tfidf
SVM	Implementing_svm.py	Train: 73.76 % Test: 71.77% Kaggle: svm.csv Eval: 70%

Table 6:SVM on tfidf dataset

```
(base) ritaja@airhead ML_Project %  cd /Users/ritaja/Desktop/Spring_24/ML/Project/ML_Project ; /Users/ritaja/opt/anaconda3/bin/python /
bundled/libs/debugpy/adapter/../../debugpy/launcher 64944 -- /Users/ritaja/Desktop/Spring_24/ML/Project/ML_Project/implementing_svm.py
running svm
train Accuracy : 73.76
test Accuracy : 71.77777777777777
(base) ritaja@airhead ML_Project %
```

Figure 5: SVM on tfidf dataset

E. Ensemble

Ensemble of perceptron and Id3: I used average- perceptron to train on misc dataset and create a bag of classifiers - 200(bagging).Then they are used to predict for each row of train, test and eval to get a new dataset consisting of N rows and 200 columns of (-1,1).Ultimately, I trained the ID3 on this new dataset to find the root and used that to predict on test and eval.

Ensemble of SVM and Perceptron : I used an Ensemble of SVM and Perceptron and see implemented it on two datasets.

Algorithm	File Name & Dataset	Accuracy on data set & best parameters
Ensemble of perceptron and Id3	perceptron_tree_ensemble_working.py Dataset: misc	Kaggle: perceptron_id3_ensemble.csv Eval: 78%

Ensemble of SVM and Perceptron	perceptron_svm_ensemble_tfidf.py Dataset: tfidf	Accuracy: 77.8 C: 1 lr: 0.01 Kaggle: perceptron_svm_ensemble_tfidf.csv Eval: 71%
Ensemble of SVM and Perceptron	perceptron_svm_ensemble.py Dataset: misc	Accuracy: 78.408 C: 1000 lr: 0.1 Kaggle: perceptron_svm_ensemble.csv Eval: 79%

Table 7: Ensemble

```
(base) ritaja@airhead ML_Project %
(base) ritaja@airhead ML_Project %  cd /Users/ritaja
bundled/libs/debugpy/adapter/../../debugpy/launcher
best parameters accuracy 77.8 C: 1 lr: 0.01
(base) ritaja@airhead ML_Project %
```

Figure 6: perceptron+Id3

```
op/Spring_24/ML/Project/ML_Project/perceptron_svm_ensmble.py
(base) ritaja@airhead ML_Project %
(base) ritaja@airhead ML_Project %  cd /Users/ritaja/Desktop/Spring
bundled/libs/debugpy/adapter/../../debugpy/launcher 51448 -- /Users
best parameters accuracy 78.408 C: 1000 lr: 0.1
(base) ritaja@airhead ML_Project %
```

Figure 7: perceptron+svm

F. Neural Network Algorithm

The miscellaneous dataset is used for the Neural Network implementation and evaluation. In order to implement NN, I used 'sklearn' library in python. The 'adam' optimizer is used in the training process and different neurons and hidden layers are tried in addition to different learning rate. I tried a network with 1, 2

and 3 layers with different number of neurons. The tuned hyper-parameters and the accuracy for cross validation, train and test sets are illustrated in Table . The network with only one hidden layer outperforms the other hyper parameters with more hidden layers. This illustrates that the simpler network works better than more complicated network for this dataset.

```
 Learning rate  hiddenLayer accuracy mean
           0.1           10      0.780064
           0.1           50      0.784636
           0.1          100      0.777377
           0.1     (10, 10)      0.783722
           0.1     (50, 50)      0.786751
           0.1 (10, 10, 10)      0.784065
           0.1 (50, 50, 50)       0.78195
          0.01           10      0.787437
          0.01           50      0.784408
          0.01          100      0.783722
          0.01     (10, 10)       0.78898
          0.01     (50, 50)      0.784351
          0.01 (10, 10, 10)      0.784979
          0.01 (50, 50, 50)      0.787037
         0.001           10      0.788923
         0.001           50      0.788237
         0.001          100      0.785722
         0.001     (10, 10)      0.789723
         0.001     (50, 50)      0.784236
         0.001 (10, 10, 10)      0.787208
         0.001 (50, 50, 50)      0.785837
        0.0001           10      0.790466
        0.0001           50      0.789895
        0.0001          100      0.790581
        0.0001     (10, 10)      0.789952
        0.0001     (50, 50)       0.78938
        0.0001 (10, 10, 10)      0.789495
        0.0001 (50, 50, 50)      0.789209
Best learning rate: 0.0001
Best Cost: 100
 Best learning rate  hiddenLayer  Best accuracy
              0.0001          100       0.790581
0.7951428571428572
0.796
0.7951428571428572 0.796
```

Figure 7:Neural Network

Algorithm	File Name	Accuracy on data set misc & best parameters
Neural Network	NN.py	Accuracy: 79 C: 100 lr: 0.0001

		Kaggle submission: nn_misc_labels.csv Eval:79%

Table 8: Neural network

G. Random Forest

Using the same 'sklearn' library I was also able to implement random forest.By using StandardScaler, RandomForestClassifier, accuracy_score, I controlled the depth hyper-parameter to 7 using parameter max_depth ; the n_estimators parameter let me set the number of trees. Then the output of all the trees are used to predict the final verdict.

Algorithm	File Name	Accuracy on data set misc
Random forest	Random_forest_misc_final.py Dataset: misc	Accuracy on Train: Accuracy on Test: Kaggle: random_forest_final.csv Eval:79%

Table 9: Random Forest

IV. Conclusion

My observation says that the performance of the algorithm is highly dependent on the selected features. Feature selection is one of the most important tasks in machine leaning; for instance, the age of the defendant can be a key feature to improve the accuracy even if we have several missing data.
I implemented a feature reduction method using entropy and information gain. In future, I would like to explore more methods that are faster for feature transformation.

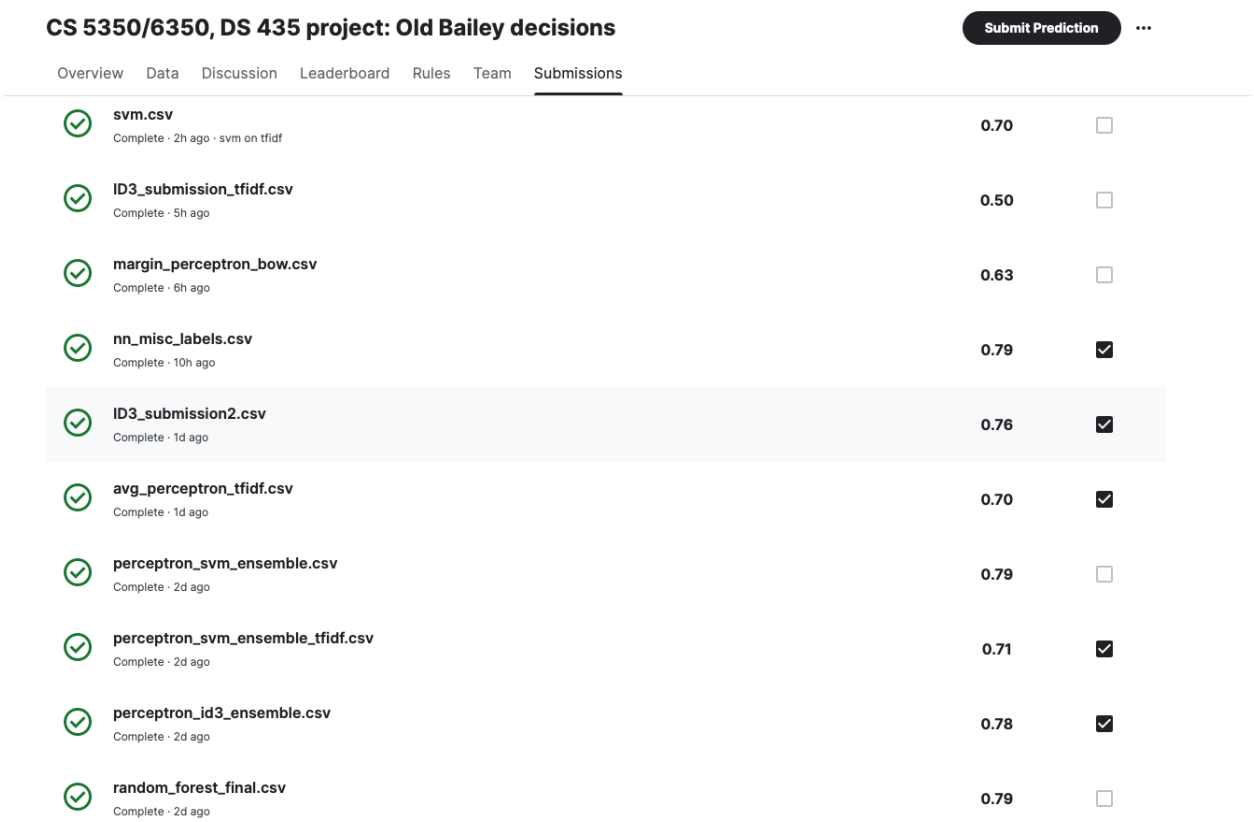

Figure 1: kaggle submissions

V. REFERENCES

[1] T. H. Clive Emsley and R. Shoemaker, “Old bailey project description,” https://www.oldbaileyonline.org/, 2020.